

EDSA Technical Architecture Brief

Automated Exam Duty & Seating Allocation System · Team of 3 · 12 weeks · 2026-09-18 ·

@Someone

What the system does

EDSA is a Java web application that turns four spreadsheets — students, rooms, faculty, timetable — into a complete exam seating plan and invigilation duty roster that satisfies every rule the examination office follows. The coordinator uploads the files, clicks once, and gets a plan back.

The whole thing runs as one Java program with no database and no external services, and is built and documented by a team of three over 12 weeks.

Documents and evidence

The code is only half of what we hand in. These are written alongside it and kept in the same repository.

- **SRS** — every requirement, numbered, so each rule in the system points back to a line someone asked for.
- **Use-case diagram** — the five things the coordinator can do, and what happens in each.
- **Class diagram** — the classes and how they connect.
- **DFD, levels 0 and 1** — where the data comes from and where it goes.
- **ER diagram** — the tables and how they relate.
- **State diagram** — the life of one plan: empty, imported, solved, edited, exported.
- **Test plan and coverage report** — what we test, and how much of the code the tests actually reach.
- **GitHub history** — commits, pull requests and reviews showing who did what and when.
- **Effort log** — hours per person per week against the 12-week plan.

Most of this comes out of working normally: the GitHub history writes itself, the coverage report is one command, and each diagram is drawn once that part of the code works. Written this way it takes about an hour a week each, which is why we do not leave it to the end.

Technology stack

Technology	Used for
Java 17 (LTS)	The whole application
Spring Boot + Thymeleaf, or servlets on embedded Jetty	HTTP layer and the five pages
HTML + minimal CSS	Page templates, server-rendered
MySQL	Storing students, rooms, faculty, timetable, saved plans and duty history
JDBC	All database access, written as plain Java
CSV + <code>BufferedReader</code>	Loading the four input files into the database
Threads + <code>synchronized</code>	Running the solver off the request thread, background rule checking
Maven	Build, dependencies, test execution
JUnit 5	Unit and integration tests
JaCoCo	Statement and branch coverage reports
Git + GitHub	Version control, pull requests, code review
GitHub Actions	Continuous integration
draw.io	UML, DFD, ER and state diagrams

Not used

No ORM and no security framework. Database access is written directly in JDBC rather than through Hibernate or JPA, and the prototype runs on the college network with no login system. The missing security layer is recorded as a scope limitation in the SRS.

How the code is organised

The code sits in three folders, each with one job. `edsa.web` shows the pages, `edsa.data` reads and writes files, and `edsa.core` makes all the decisions. Arrows point one way only: the pages may use the core, but the core never uses the pages.

```
flowchart TD
    B[Browser] --> W["edsa.web<br/>controllers, templates"]
    W --> D["edsa.data<br/>CSV, reports, saved plans"]
    W --> C["edsa.core<br/>model, rules, solver"]
    D --> C
    C -. -> X["no imports upward"]
```

That one-way rule is the reason the decision-making can be tested without opening a browser, and the reason the pages could be replaced later without touching the rules.

edsa.core — the brain

- **The things:** `Student`, `Faculty`, `Room`, `ExamSlot`, `Seat`, `SeatingPlan`, `DutyRoster`, `Assignment`, `WorkloadLedger`, and `Person` as a shared parent of `Student` and `Faculty`.
- **The rules:** a `Constraint` interface, split into hard and soft, with one class per rule and a `PlanChecker` that runs them all and lists what is broken.
- **The decisions:** `GreedyAllocator` makes a first plan, `LocalSearchImprover` makes it fairer, `AbsenceRepairer` handles a missing invigilator, and `ValidatorThread` keeps checking the plan in the background.
- **The errors:** four custom exceptions for capacity, broken rules, no substitute available, and bad input files.

edsa.data — the files

Reads the four CSVs and checks every row, writes the reports out, loads them into the database, saves and reloads finished plans, and keeps a log of what the coordinator did.

edsa.web — the five pages

Page	What it does
Import	Upload the four CSVs, show row counts and malformed-row warnings

Page	What it does
Solve	Start the run, show progress and the violation count falling to zero
Plan viewer	Seating and duty tables, filter by room or date
Disruption	Mark a faculty member absent, show a diff of exactly what the repairer changed
Reports	Preview and download seating charts, attendance sheets, workload summary

One rule for this folder: pages never decide anything. A page takes the request, asks the core, and shows the answer. Any rule about seniority or capacity written inside a page is in the wrong folder.

One complete run

From the coordinator opening the site to a downloaded seating chart, with the folder doing the work named at each step.

1. **Upload.** The page takes the four CSV files and hands over their locations. It does not read them itself.
2. **Read the files.** `data` reads each file line by line and checks every row, then builds the student, room, faculty and slot objects. A bad row is reported with the file name and line number instead of being skipped quietly.
3. **Sort into memory.** `core` puts everything into the collections that make lookups fast: a map to find anyone by ID, a sorted set to keep faculty in order of how many duties they have, a queue to always pull the hardest slot next.
4. **First plan.** `core` takes the hardest slot first, seats the students on the room grid, and puts invigilators on duty. Every choice is checked against the hard rules before it is accepted. The plan is now legal, but the duties are uneven.
5. **Make it fairer.** `core` swaps two duties, scores the result, and keeps the swap only if the score improved. It repeats until nothing improves or time runs out. The same input always gives the same output — nothing is learned or remembered between runs.
6. **Show and save.** The pages display the plan as tables. `data` saves the whole plan to the database, writes to the log, and updates the duty history so next time's fairness starts from what really happened.

7. **Keep checking.** A background thread re-checks the plan after any manual edit, so a broken rule is caught on screen instead of in the exam hall.
8. **Someone is absent.** Mark them unavailable, and `core` finds a legal replacement while moving as few other people as possible — or says clearly that no replacement exists. The page shows what changed.
9. **Reports.** `data` builds the seating charts, attendance sheets and workload summary, and the page offers them as downloads.

Steps 4 and 5 can take a while, so they run on a separate thread. The Solve page starts the job and then keeps asking for progress, so the browser never sits waiting for the solver to finish.

Rules and scoring

There are two kinds of rules. **Hard rules** decide whether a plan is allowed at all — anything that breaks one is thrown away immediately. **Soft rules** are preferences, and each one adds penalty points when it is not met well. Adding up those points gives the plan a score, and a lower score is a better plan. That one number is the system's whole idea of "better".

Hard constraints: no invigilator on their own subject's paper, no double-booking of a person or room, room capacity never exceeded, no two students of the same paper seated adjacent.

Soft constraint	Penalty	Why weighted this way
Workload fairness	$(\text{duties} - \text{fair share})^2$, per invigilator	Squaring punishes one badly overloaded person far more than several being slightly off: 6/3/2/1 scores 14, while 4/3/3/2 scores 2
Consecutive slots	5 per back-to-back pair	Tiring but sometimes unavoidable, so discouraged rather than forbidden
Seniority cover	10 per room with no senior	Highest of the soft weights: a room without an experienced invigilator carries real operational risk
Department spread	3 per single-department room	Mild preference; convention rather than requirement

These weights encode the examination office's priorities, not yours. They come from the requirements interview and live in one configuration class, so the ordering changes without touching the solver.

Adding a constraint later

This is the part of the design that makes incremental work possible, and it is worth stating explicitly in the SRS. Each new rule is:

1. One new class implementing `Constraint`, in `edsa.core`.
2. One JUnit test class proving it rejects what it should and accepts what it should.
3. One line registering it in the constraint list, plus a weight if it is soft.
4. One pull request.

Nothing in the solver, the storage layer or the web layer changes. If adding a rule ever requires editing `GreedyAllocator`, the abstraction has leaked and that is worth investigating before writing more rules.

How the three of you work

Each member owns a vertical slice, so everyone writes engine, storage and web code. Splitting by layer would leave each person understanding a third of the system and push all integration risk into the final weeks.

Member	Owns	Writes code in
1	Data and rules	Domain classes, CSV import, serialization, the six rule classes, <code>PlanChecker</code> , plus the Import page
2	The solver	<code>GreedyAllocator</code> , <code>LocalSearchImprover</code> , <code>AbsenceRepairer</code> , <code>ValidatorThread</code> , plus the Solve and Disruption pages
3	Presentation and platform	Plan viewer and Reports pages, report generation, Maven, GitHub Actions, JUnit harness, profiling

Whoever owns a slice also writes its tests and its share of the SE documents. Rotate who reviews whom so all three names appear on reviews.

GitHub settings

Your repository is public, so branch protection is enforceable on the free plan — the notice on that settings page says the free plan can only enforce rules on public repositories. If you later make the repo private, these rules stop applying and the discipline becomes voluntary.

On the branch protection page, set the branch name pattern to `main` and tick:

- **Require a pull request before merging**, with 1 required approval. With three of you, this means every change is read by someone else — the review evidence UCS503 wants.
- **Require status checks to pass before merging**. Add the GitHub Actions check here after your first CI run, since the check name only appears in the list once it has run at least once.
- **Require conversation resolution before merging**, so review comments cannot be ignored silently.

Leave these alone: signed commits (setup cost, no marks), linear history (optional), require deployments, lock branch, allow force pushes, allow deletions. Leave **Do not allow bypassing the above settings** unticked for now, so the repo owner can rescue a stuck merge the night before a deadline.

The loop

1. `git checkout -b feature/csv-import` — one branch per task, named for the task.
2. Commit in small steps with messages that say what changed and why.
3. Push and open a pull request; CI runs the full suite on it.
4. Another member reviews, comments, approves.
5. Merge to `main`. Merge at least weekly, even if a feature is unfinished behind a flag.

Never push straight to `main`, and do not let one person merge their own work. Both rules exist for the SE grade as much as for the code.

Build, run, and the 12-week order

```
mvn clean test          compile and run every test with a JaCoCo coverage
report
mvn spring-boot:run     start the web application (or mvn exec:java for the
Jetty option)
java -cp target/classes edsa.core.CliRunner students.csv rooms.csv
faculty.csv timetable.csv
```

The third command drives the engine headlessly. It is how the solver is tested and demonstrated while the pages are still being built, and it works because `edsa.core` has no web dependency.

Milestone order

The sequencing rule: something runnable exists from week 3 onward, and every later week adds one thing to a working system rather than assembling parts at the end.

Weeks	Deliverable
1-2	SRS, use cases, requirements interview. Repo, Maven skeleton, CI green on an empty test
3	Domain classes and CSV import, with tests
4	Class diagram, DFD, ER diagram matching the code as written
5	Greedy allocator with room capacity and double-booking only. Runs from <code>CliRunner</code>
6	First web pages: Import and Plan viewer. End-to-end run in a browser — this is the prototype
7	Remaining hard constraints, one per pull request
8	Soft constraints and penalty scoring
9	<code>LocalSearchImprover</code> and the Solve page with progress polling
10	<code>AbsenceRepairer</code> , <code>ValidatorThread</code> , the Disruption page

Weeks	Deliverable
11	Reports, downloads, saving and reloading plans from the database
12	Test plan writeup, coverage report, profiling, final documents, demo rehearsal

Build CI in week 1, not week 12. A test suite added at the end proves nothing about the weeks it was absent, and the commit history shows exactly when it appeared.